

DOMANDA 1

Devo ridurre i conflitti sulla stessa struttura offrendo due copie di tags, una per offrire il tag in caso di miss della cache di livello uno, una per rispondere alle operazioni effettuate sul bus condiviso. Se ho due copie, devo aggiornarle quando voglio cambiare queste due copie.

Questo è un problema?

L'intervallo di tempo in cui devo cambiare i due valori del tag, potrebbe causare un effetto collaterale sulla cache di primo livello o sul bus condiviso.

Il problema è che l'attività tradizionale del secondo livello di cache è risolvere le miss del primo livello e per cercare quale blocco caricare bisogna cercare attraverso il tag.

Quando devo aggiornare i tag devo eseguire un'azione atomica per evitare inconsistenze.

Questa update può essere un problema dal punto di vista delle performance del processore?

Sì, infatti ho una perdita di performance.

Quando devo aggiornare entrambe le copie del tag?

La condizione per la quale devo aggiornare entrambi i tag è nel caso di miss nella cache di secondo livello, ma quando lo faccio uso il bus condiviso per chiedere una copia del blocco coinvolto nella miss, quindi il bus è occupato da un'operazione richiesta dalla cache di secondo livello, quindi non c'è un drop delle performance.

DOMANDA 2

Che side effect potrebbe avere la inclusion property?

Inclusion property → mi dice che le cache di livello superiore (l3) devono avere una copia dei blocchi delle cache di livello inferiore (l2, l1), ma le cache di livello inferiore (l1, l2) non devono avere una copia dei blocchi delle cache di livello superiore (l3). Se questo è vero, allora la dimensione della cache è pari alla dimensione dell'ultimo livello, e non alla somma delle dimensioni di tutti i livelli.

Nell'ultimo microprocessore intel la

inclusion property non è garantita perché può portare a problemi dal punto di vista energetico. Il problema è che se i tre livelli di cache devono avere gli stessi blocchi, la dimensione massima della cache è data dall'ultimo livello di cache, quella più piccola e questo può peggiorare le performance. La soluzione è l'uso di una victim cache che permette di usare una copia locale senza richiederla attraverso il bus condiviso.

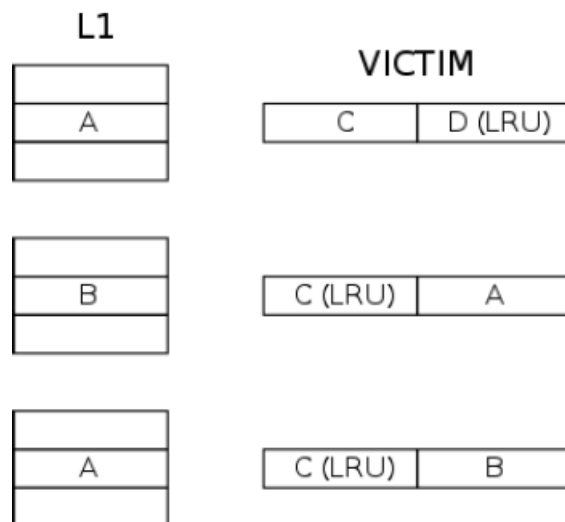
La inclusion property è usata per ridurre la complessità del sistema ma questo può portare a problemi di performance.

Il problema di performance è dovuto al fatto che ogni volta che si esegue una scrittura questa debba essere propagata sui livelli successivi.

Come si può risolvere?(SBAGLIATA)

Una victim cache è una cache hardware progettata per ridurre i conflitti nelle miss e migliorare la latenza delle hit per le cache direct-mapped. Viene impiegato nel percorso di riempimento di una cache di livello 1, in modo tale che qualsiasi blocco che viene espulso dalla cache venga memorizzato nella victim cache. Pertanto, la victim cache viene popolata solo quando i dati vengono espulsi dalla cache di livello 1. In caso di miss nel Livello 1, la voce mancante viene cercata nella victim cache. Se l'accesso risultante è un hit, i contenuti del blocco della cache di livello 1 e del corrispondente blocco della cache della vittima vengono scambiati.

La victim cache serve per ridurre la miss di conflitto, è una full-associative molto piccola posizionata vicino alla L1 e serve per ridurre le cache di conflitto su L1, aumentando hit rate.



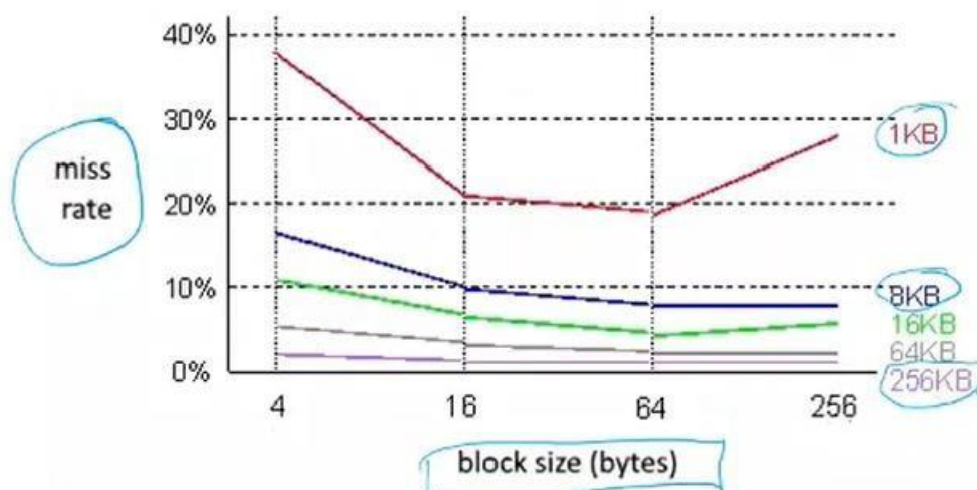
DOMANDA 3

Perché all'interno di un multiprocessore non ho problemi di workload-balancing?

Il workload bilanciato può essere ottenuto automaticamente tramite lo scheduler che offre un task diverso ad ogni processore. L'unico problema è riuscire a fornire un numero di processi pronti tale da fare operare il sistema alla massima utilizzazione e minimizzare la comunicazione tra processi. In tal caso l'overhead è dovuto alla comunicazione tra processi (shared memory e cache)

DOMANDA 4

Andamento miss rate in base alla grandezza del blocco. Vario la dimensione del blocco in bytes (4-16-64-256) e la dimensione della cache in kB (1-8-16-64-256). Perché ha quell'andamento? (domanda tipica)



In generale la miss rate va giù quando la dimensione del blocco aumenta.

Quindi, all'aumentare della dimensione del blocco il miss rate diminuisce.

Nel caso della cache da 1 kB invece la miss rate aumenta. Questo andamento negativo è dovuto alla località temporale. (se aumento la dimensione dei blocchi, diminuisce il numero di blocchi che posso memorizzare)

Sono possibili due soluzioni, una risulta essere migliore nel caso di località spaziale, mentre l'altra nel caso di località temporale.

Nel caso di località temporale voglio avere blocchi piccoli (4 bytes) perché voglio caricarne il maggior numero possibile.

Nel caso di località spaziale vogliamo avere la porzione più grande di memoria possibile attorno all'indirizzo della miss. (massimizzare la dimensione del blocco)

Tornando al caso della cache da 1 KB, in caso di blocchi da 256 bytes avrò solamente 4 blocchi in cache, che è la migliore situazione per la località spaziale, ma decisamente la soluzione non ottimale per la località temporale.

Nel caso di blocchi da 4 bytes avrò 256 blocchi.

Dobbiamo quindi cercare la dimensione ottimale che solitamente è vicina alla dimensione di una word in cache moltiplicata per quattro.

Soluzione = $\text{sizeof}(\text{word}) * 4$.

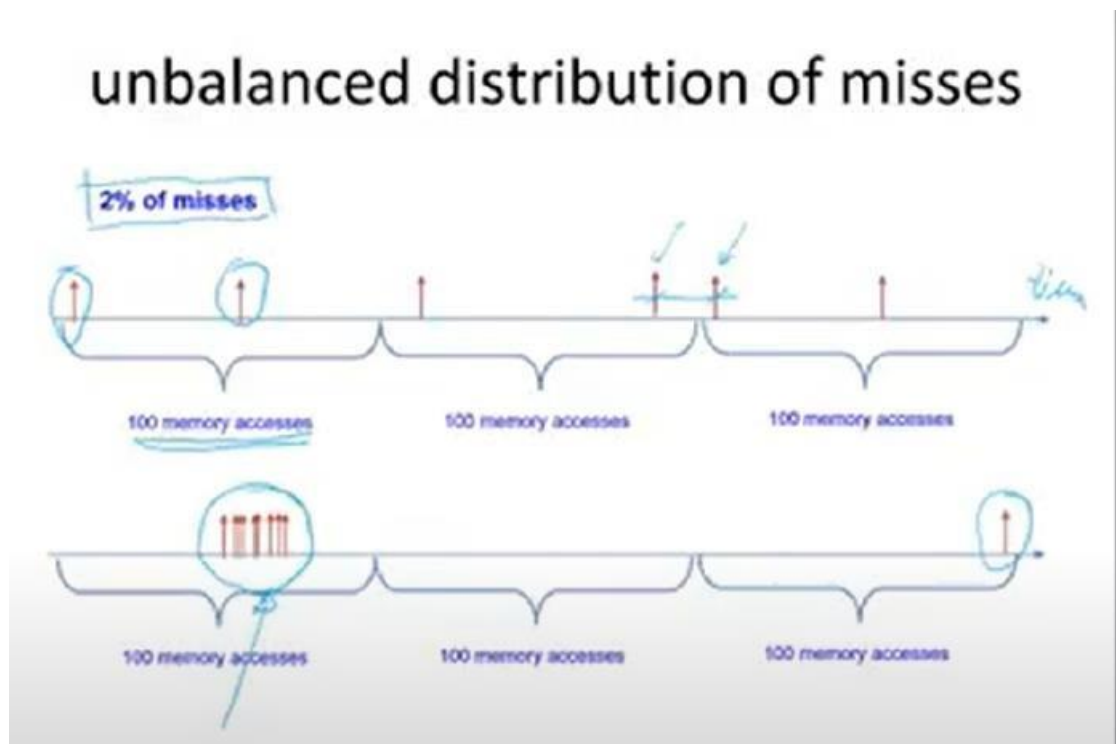
Il valore ottimo però potrebbe dipendere da applicazione a applicazione, dato che cambia in base al tipo di accessi che farà l'applicazione in cache.

DOMANDA 5

Perché potrebbero esserci molte miss in un intervallo molto ristretto di tempo e una molto dopo?

La distribuzione delle miss non è uniforme. Infatti, le miss sono generalmente concentrate all'arrivo del processo, quando la sua immagine non è ancora in cache.

chiamate di funzioni (call), cambio di contesto tra processi, interrupt (int)



DOMANDA 6

La memoria è logicamente divisa in varie aree. Quali sono le possibili aree nelle quali avrò gli accessi? (legata a quella precedente)

- main code
- global variables
- stack (local variable)
- codice di funzioni: codice usato per chiamare le librerie, funzioni I/O
- codice del kernel

se queste aree nella cache non sono sovrapposte, allora non si verificano miss. Se invece sono sovrapposte, allora si verificano miss.

Conclusioni che possiamo trarre?

Come posso ridurre il numero di miss?

Una buona idea è separare le cache per i dati e le cache per il codice.

2 cache, una per il codice e una per i dati.

Da una parte posso leggere i dati, dall'altra fare il fetch di una nuova istruzione. Posso aumentare il parallelismo tra l'accesso ai dati e l'accesso al codice.

Possiamo organizzare questi due livelli di cache in modi diversi, dato che i meccanismi di località spaziale e temporale delle cache dei dati e del codice sono molto diversi.

Un esempio potrebbe essere quello di utilizzare una set associative cache a 4 vie: una via per l'area istruzioni, una via per lo stack, una via per le variabili globali e una via per il kernel e le funzioni di libreria.

Direct-mapped cache o set-associative?

Le set associative ci permettono di avere un minor miss rate, mentre aumenta il tempo per le hit.

Quindi se si vuole diminuire il numero di miss vanno utilizzate le set associative.

Set-associative, potrebbe esserci dei conflitti dati dal fatto che potrebbero essere eseguite delle istruzioni contenute nello stesso blocco.

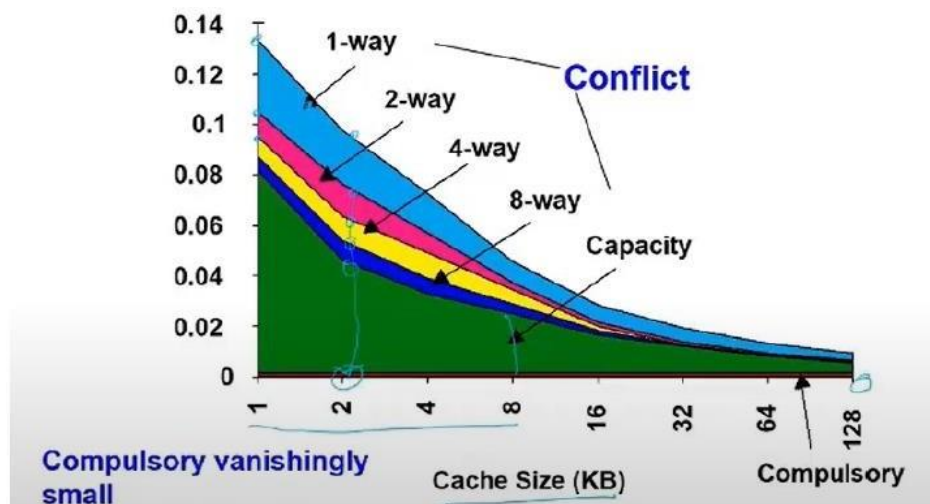
Quanti blocchi per set?

4-way associative cache, perché altrimenti un loop (grande) potrebbe eliminare le copie in cache appartenenti allo stesso loop.

Le migliori performance si ottengono con una cache a 4 vie in modo da avere una via per ogni area come detto prima.

DOMANDA 7

Come possiamo valutare la distribuzione delle miss per una cache quando il processore sta eseguendo un particolare programma, riuscendo ad individuare la causa della miss?



Una miss può essere dovuta a tre possibili cause:

compulsory → primo accesso ad un blocco. Si potrebbe risolvere aumentando la grandezza dei blocchi, ma il numero di miss dovuto a compulsory è basso, per questo non ha senso intervenire per risolverlo.

Capacity → la cache non può contenere tutti i blocchi acceduti dal programma. In tal caso la soluzione consiste nell'utilizzare una cache più grande (questo potrebbe aumentare il tempo di accesso alla cache)

Conflict → più locazioni di memoria sono mappate nella stessa locazione della cache. In tal caso sono possibili due soluzioni: aumentare la dimensione della cache, aumentare il numero di vie di una cache associativa.

In generale, il numero di miss dipende dalla stessa capacità della cache.

DOMANDA 8

Perché i microprocessori attuali usano una cache ad indirizzi fisici al posto di una ad indirizzi virtuali?

Perché due indirizzi virtuali diversi possono mappare lo stesso indirizzo fisico (alias), quindi lo stesso blocco di cache, che invece è univoco.

Ovvero, le pagine fisiche possono essere condivise tramite due indirizzi virtuali che puntano allo stesso indirizzo fisico, questo serve per permettere a programmi diversi di condividere porzioni di dati o di codice (ovvero la stessa pagina fisica).

Quindi, per ogni accesso in memoria bisogna considerare due operazioni: la traduzione dell'indirizzo e l'accesso in cache. Pertanto, uso la MMU per tradurre l'indirizzo virtuale in indirizzo fisico, e uso l'indirizzo fisico per accedere in cache.

Per poter utilizzare gli indirizzi virtuali la cache dovrebbe poter prevedere gli alias (ovvero più indirizzi virtuali possono corrispondere allo stesso indirizzo fisico). Questa complessità non si può avere nella cache di livello 1, che deve essere molto veloce per gestire le richieste della CPU. Per le cache di livello superiore invece non abbiamo questo problema perché la traduzione è stata già effettuata per accedere al primo livello di cache.

Pertanto per accedere alla L1 devo attendere il TLB (cache che mantiene la traduzione degli indirizzi). Una soluzione per velocizzare il processo potrebbe essere quella di parallelizzare le due parti, quindi attendere che gli indirizzi diventino stabili. Questo può essere fatto in due modi:

- Hardware → uso una linea di bit del TLB data per indirizzare la cache tag, quindi basta aggiungere un bit addizionale da TLB data a L1 cache tag. In pratica, questo significa che quando il processore effettua una richiesta di memoria, il TLB restituisce l'indirizzo fisico corrispondente all'indirizzo virtuale richiesto, insieme a una linea di bit che viene utilizzata per selezionare la corretta linea di cache nella cache di primo livello (L1). Aggiungere un bit addizionale da TLB data a L1 cache tag può ridurre il tempo di accesso alla cache di primo livello, in quanto l'indirizzo fisico viene utilizzato per selezionare la corretta linea di cache e il bit aggiuntivo viene utilizzato per selezionare il corretto blocco all'interno di quella linea di cache.
- Software → faccio in modo che l'ultimo bit del numero di pagina virtuale e l'ultimo bit di pagina fisica siano uguali. Questa ottimizzazione viene chiamata "pagina allineata ai confini di cache". In pratica, quando il sistema effettua la traduzione degli indirizzi di memoria, il bit meno significativo dell'indirizzo virtuale viene utilizzato per selezionare la linea di cache corretta nella cache di primo livello. In questo modo, il sistema evita di dover effettuare una ricerca nella cache di traduzione degli indirizzi (TLB), accelerando l'accesso alla cache di primo livello.

DOMANDA 9

Ad un cambio di contesto posso avere molte miss nello stesso momento, come posso evitarlo?

Il problema si verifica quando due o più processori hanno un context switch nello stesso momento. Per risolvere quindi basta non permettere di avere cambi di contesto nello stesso momento. Quindi, il sistema operativo deve avere uno scheduler che evita l'accavallarsi dei context switch su più cpu contemporaneamente.

DOMANDA 10

Come possiamo implementare a livello hardware la mutua esclusione?

Come si può garantire la mutua esclusione in modo atomico?

La mutua esclusione può essere implementata attraverso l'utilizzo di lock e unlock.

Dobbiamo, però, distinguere il caso in cui si parla di un sistema monoprocesso, da quello multiprocesso.

MONOPROCESSORE:

Nei sistemi con un solo processore, un modo per implementare la lock è quello di disabilitare le interruzioni. In questo modo il processo che esegue una sezione critica non può essere interrotto.

MULTIPROCESSORE:

Disabilitare le interruzioni risulta essere però troppo inefficiente per i sistemi multiprocessori (queste dovrebbero essere disabilite su tutti i processori).

Per questo motivo, si usano invece istruzioni hardware speciali che funzionano in modo atomico, come ad esempio la Test&Set.

L'istruzione test-and-set viene usata per scrivere in una locazione di memoria e restituire il suo vecchio valore come una singola operazione atomica (non interrompibile).

Se dobbiamo fare la lock e la unlock su un certo valore, senza un'istruzione singola (read, write, load, store), come possiamo fare?

Con la test & set.

Conoscete un modo per garantire che la test & set sia atomica in un multiprocesso? (in teoria è già atomica)

Il bus è gestito da un controllore (arbitro) che deve selezionare per ogni operazione il processore che può usare il bus.

Un modo per risolvere il problema è bloccare in controllore per due operazioni per garantire l'esecuzione atomica e senza interruzioni.

(BOZZA)

Another alternative is to have a pair of instructions where the second instruction returns a value from which it can be deduced whether the pair of instructions was executed as though the instructions were atomic.

The pair of instructions is effectively atomic if it appears as though all other operations executed by any processor occurred before or after the pair.

In RISC V, the pair of instructions includes a special load called a load reserved (also called load linked or load locked) and a special store called a store conditional.

DOMANDA 11

Abbiamo visto come organizzare il lavoro in pipeline. Dal punto di vista delle performance quali sono i fattori che possono incidere sul throughput delle ALU?

[Risposta] Stalli causati da attesa di operandi, chiamati anche data hazard. Cioè, potrebbe accadere che ci sia una istruzione che abbia come operando il risultato dell'istruzione precedente, pertanto per poter essere eseguita deve aspettare che il risultato dell'istruzione precedente sia scritto nel registro:

add r1, r2, r3 $\rightarrow$ $r1 = r2 + r3$

sub r4, r5, r1 $\rightarrow$ $r4 = r5 - r1$

La soluzione può essere usare delle "scorciatoie" a livello hardware, ovvero attraverso dei circuiti di bypass, che riportano l'uscita dello stage di esecuzione all'ingresso del registro che precede lo stage di lettura degli operandi.

Se non usassi questa cosa dovrei inserire uno stallo e aspettare che il risultato della prima istruzione venga scritto in memoria o nel registro di destinazione.

Dopo ha voluto anche sapere il register renaming.

Un altro possibile modo per risolvere gli stalli è quello dell'esecuzione fuori ordine, ovvero eliminare le situazioni di stallo eseguendo le istruzioni in un ordine diverso da quello previsto dal programma. In tal caso, bisogna fare attenzione alle dipendenze sui nomi, che si verificano quando due istruzioni scrivono nello stesso registro.

Add r1, r2, r3 $\rightarrow$ r1=r2+r3

Sub r2, r4, r5 $\rightarrow$ r2=r4-r5

Mul r6, r2, r1 $\rightarrow$ r6=r2*r1

In tal caso la sub non usa il risultato della add, ma non può essere eseguita prima della add perché modifica il contenuto di r2 che è letto dalla add $\rightarrow$ dipendenza sui nomi

Le dipendenze sui nomi possono essere eliminate tramite la rinomina dei registri. Quindi, nell'esempio di prima, il risultato della sub può essere scritto in un altro registro non utilizzato (come r7), purché venga modificata anche l'istruzione successiva mul che deve leggere il risultato della sub (non più r2, ma r7)

Add r1, r2, r3 $\rightarrow$ r1=r2+r3

Sub r2, r4, r5 $\rightarrow$ r7=r4-r5

Mul r6, r2, r1 $\rightarrow$ r6=r7*r1

I registri del processore si dividono in architetturali (registri che il compilatore può usare) e fisici (registri della cpu), quindi in fase di esecuzione i registri architetturali devono essere rinominati e mappati nei registri fisici.

Come ci aiuta la cache in questa situazione?

Recupero i dati più velocemente.

ALTRA DOMANDA

La presenza delle cache introduce vari problemi, ma uno di questi è particolarmente interessante e influenza lo sviluppo del sistema operativo in particolare il multicore, quale? Cache coherence, perché la studiamo? Perché è un bottleneck? Perché è molto più utilizzata?

Il problema della cache coherence si presenta in sistemi in cui sono presenti varie unità di elaborazione che condividono lo stesso spazio di memoria.

Naturalmente, allo scopo di ottimizzare le prestazioni di tali sistemi, si introducono delle cache articolate in più livelli. Si viene così a creare una gerarchia di memorie, alcune delle quali condivise, altre private.

È necessario mettere a disposizione dei processori un meccanismo che consenta di mantenere in ogni istante la correttezza delle informazioni contenute nelle varie cache del sistema, in modo tale che i processori abbiano sempre la stessa visione dello spazio di memoria.

Detto in termini informali, il meccanismo che gestisce la cache coherence deve far sì che ogni lettura di una certa variabile restituisca il valore scritto più di recente per quella variabile.

I protocolli di cache coherence si occupano di mantenere la coerenza dei dati nei blocchi di cache condivisi, cercando di limitare il più possibile la latenza dovuta alla generazione di cache miss dovute alle operazioni di scrittura.

Perché il problema fondamentale della cache sono le write. (slide su read e write cache coherence)

Perché le miss delle letture sono mitigate dall'architettura multilivello della cache, mentre le scritture devono comunque essere fatte in memoria prima o poi.

Quindi per migliorare le prestazioni dobbiamo diminuire il numero di scritture.

DOMANDA 12

Perché su un multiprocessore abbiamo introdotto più linee di pipeline? E perché non troviamo processori con 15 pipeline?

[Risposta] Ovviamente aumentiamo linee di pipeline perché aumenta throughput, il problema di averne troppi è che devo accelerare programmi sequenziali per quello che riguarda la CPU. Dovrei aumentare il numero di istruzioni parallele in programmi sequenziali, mentre la maggior parte dei programmi che abbiamo sono sequenziali.

DOMANDA 13

Modelli di multithreading per i multiprocessori.

- Coarse-grained multithreading → ogni volta che un thread causa uno stallo nella pipeline, faccio un thread switching
- Fine-grained multithreading → i thread si intervallano sempre secondo uno scheduling round-robin. Se ho un thread che non causa stalli, viene comunque eseguito il thread switching, pertanto questo dovrà aspettare il proprio turno per finire la propria esecuzione
- SMT → è un'evoluzione del fine-grained. Ad ogni istante posso eseguire qualsiasi istruzione relativa a qualsiasi thread, in qualsiasi functional unit.

Quali sono i vantaggi dell'avere multithreading.

Con il multithreading più thread condividono le stesse functional unit. In questo modo si ottiene una migliore utilizzazione delle risorse, facendo sì che il tempo in cui le risorse sono libere sia minore.

Il vantaggio del SMT è che permette di sfruttare meglio le risorse del processore, riducendo il tempo di inattività del processore quando ci sono pochi thread da eseguire. Inoltre, il SMT può aumentare la capacità di risposta del sistema, in quanto consente di eseguire più applicazioni contemporaneamente senza compromettere le prestazioni del sistema.

Tuttavia, l'utilizzo del SMT può comportare anche alcuni svantaggi. Ad esempio, l'utilizzo di più thread logici può aumentare il consumo di energia e la temperatura del processore, in quanto le unità funzionali del processore sono più utilizzate. Inoltre, il SMT può causare una riduzione delle prestazioni in alcune applicazioni, in quanto la condivisione delle risorse del processore tra i thread logici può causare una congestione delle risorse.

In sintesi, il SMT è una tecnologia utile per aumentare l'utilizzo del processore e migliorare la capacità di risposta del sistema. Tuttavia, l'utilizzo del SMT deve essere valutato attentamente in base alle esigenze specifiche del sistema e alle prestazioni richieste.

DOMANDA 14

Come si possono ridurre le conflict miss in cache? Poi è passato a chiedere cosa sono le conflict misses.

[Risposta] Aumentare l'associatività. Infatti, aumentando l'associatività, aumento il numero di blocchi che posso memorizzare in cache per ogni indice (se ho una cache a 4 vie posso memorizzare 4 blocchi per ogni indice).

I conflict miss si verificano quando una miss causa il rimpiazzamento di un blocco presente in cache.

DOMANDA 15

Solitamente abbiamo il primo livello di cache piccolo, i successivi grandi, perché?

[Risposta] il primo livello di cache deve essere veloce per gestire le richieste del processore nel minor tempo possibile, e piccolo per mantenere bassa la complessità. Per questo motivo si usano cache di primo livello piccole che sono anche più veloci.

Il secondo livello di cache viene introdotto per risolvere le miss del primo livello di cache. Quindi possono essere più lente del primo livello e più grandi, in modo da limitare il più possibile le miss. Inoltre, si usano cache più lente, e quindi più grandi, per limitare i costi.

Perché vogliamo minimizzare l'access time del primo livello di cache, non ci importa avere un grande numero di hit. Invece i livelli successivi possono avere access time più alti ma devono minimizzare il numero di miss.

DOMANDA 16 == domanda 8

Come funziona la cache di primo livello in combinazione con la traduzione degli indirizzi?

Il primo livello di cache usa indirizzi fisici o virtuali?

[Risposta] usa indirizzi fisici, perché per gli indirizzi virtuali ho un problema.

Infatti, più di un indirizzo virtuale può riferire lo stesso indirizzo fisico e per questo motivo vado ad usare direttamente gli indirizzi fisici, in modo da evitare di dover aggiungere complessità al primo livello di cache.

Per poter utilizzare gli indirizzi virtuali la cache dovrebbe poter prevedere gli alias (ovvero più indirizzi virtuali possono corrispondere allo stesso indirizzo fisico). Questa complessità non si può avere nella cache di livello 1, che deve essere molto veloce per gestire le richieste della CPU. Per le cache di livello superiore invece non abbiamo questo problema perché la traduzione è stata già effettuata per accedere al primo livello di cache.

DOMANDA 17

Perché si usano due cache di livello 1, una per le istruzioni e una per i dati?

[Risposta] Nella pipeline l'accesso ai dati e alle istruzioni avviene in stage diversi e con frequenze differenti. Per questo motivo si usano due cache di primo livello separate per istruzioni e dati. In questo modo lo stage di fetch (che accede alle istruzioni) e lo stage di lettura degli operandi (che accede ai dati) possono eseguire in contemporanea.

Inoltre, se uso cache diverse riduco il numero di miss.

DOMANDA 18

Dal punto di vista del consumo energetico è più importante l'energia o la potenza dissipata?

Che importanza ha controllare il picco massimo di potenza di un processore?

L'energia dissipata è quella che riscalda il nostro sistema. Il picco di potenza dà fastidio perché $P=VI$, se accendo contemporaneamente 4 vie devo fornire corrente e in questo momento ho un picco di potenza dato dalla corrente di 4 vie accese. Si crea un *hotspot*, picco di potenza localizzato e breve che non si può controllare con la ventola o un altro di sistema di raffreddamento. I sistemi embedded quindi devono mantenere bassa sia la potenza che l'energia.

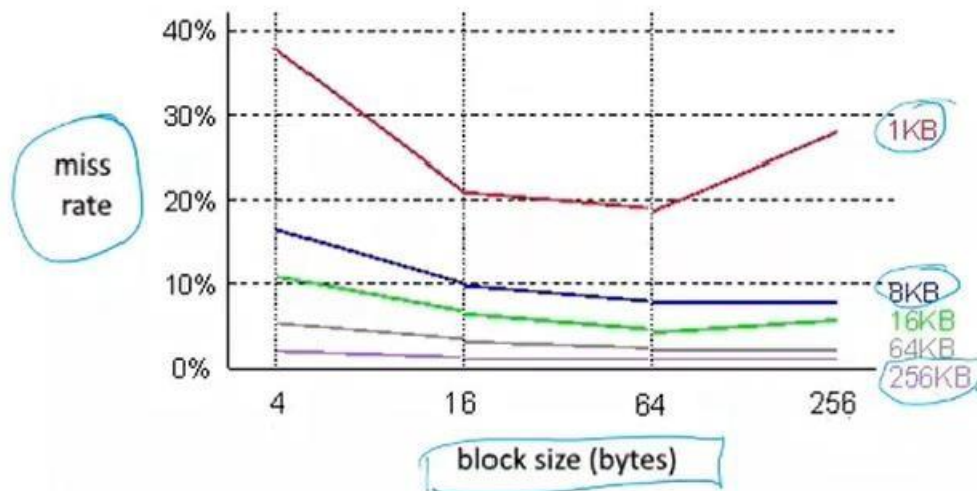
DOMANDA 19 (simile alla domanda 4)

Andamento del tempo di esecuzione totale al variare della dimensione del blocco con capacità della cache.

[Risposta] In generale la miss rate va giù quando la dimensione del blocco aumenta.

Quindi, all'aumentare della dimensione del blocco, il miss rate diminuisce.

Se il miss rate diminuisce, diminuisce anche il tempo di esecuzione, visto che ci vorrà meno tempo per accedere in memoria, visto che i dati sono contenuti in cache.



Nel caso della cache da 1 kB, invece, la miss rate aumenta all'aumentare della dimensione del blocco. Questo andamento negativo è dovuto alla località temporale. (se aumento la dimensione dei blocchi, diminuisce il numero di blocchi che posso memorizzare)

Sono possibili due soluzioni, una risulta essere migliore nel caso di località spaziale, mentre l'altra nel caso di località temporale.

Nel caso di località temporale voglio avere blocchi piccoli (4 bytes) perché voglio caricarne il maggior numero possibile.

Nel caso di località spaziale vogliamo avere la porzione più grande di memoria possibile attorno all'indirizzo della miss. (massimizzare la dimensione del blocco)

Voleva sapere località temporale e spaziale.

Località spaziale → se un programma accede ad un certo indirizzo di memoria, è probabile che in breve tempo accederà ad un indirizzo vicino

Località temporale → se un programma accede ad un certo indirizzo di memoria, è probabile che in breve tempo vi accederà di nuovo

DOMANDA 20

Quale è il metodo più veloce che possiamo avere per capire le prestazioni di un sistema?

[Risposta] Tool di debug per trovare il bottleneck, trovo la componente più utilizzata (ovvero la componente in cui i programmi passano la maggior parte del tempo), faccio un benchmark su queste componenti.

DOMANDA 21

A partire da una organizzazione funzionale di una CPU siamo arrivati ad una organizzazione a pipeline, perché la pipeline mi permette di aumentare le prestazioni e come faccio a vedere che effettivamente aumentano?

Pipeline → l'esecuzione di una singola istruzione viene suddivisa in più fasi, ciascuna delle quali impegna un'unità del processore. Quando un'istruzione ha terminato una fase, libera l'unità del processore che può quindi essere usata per fare avanzare un'altra istruzione. Così facendo le varie fasi possono essere eseguite in parallelo su istruzioni diverse, quindi possiamo sovrapporre nel

tempo l'esecuzione di più istruzioni. In questo modo aumento anche l'utilizzazione delle unità del processore.

Quindi, la pipeline mi consente di aumentare le prestazioni perché posso iniziare ad eseguire un'istruzione anche se la precedente non ha ancora finito la sua esecuzione, ovvero posso sovrapporre nel tempo l'esecuzione di più istruzioni.

La metrica che mi fa capire che le prestazioni aumentano è l'instruction throughput, ovvero il numero di istruzioni completate per secondo.

[Risposta] Aumento utilizzazione dividendo le fasi di utilizzo della CPU e potendo iniziare ad eseguire un'istruzione prima che l'ultima sia completamente finita. Il parametro che posso guardare è il numero di operazioni committate al secondo (per esempio). Devo mantenere cariche quante più ALU occupate possibile.

DOMANDA 22*

Quali sono le soluzioni che si possono utilizzare per un processore vettoriale?

Processore vettoriale → esegue un'istruzione su un vettore di dati

Una possibile soluzione è quella di utilizzare delle pipeline in parallelo.

Quali sono i punti di sovrapposizione con la GPU? Boh!!

Uso della cache in questo tipo di architettura?

I processori vettoriali possono avere cache. Le cache di istruzioni sono abbastanza comuni. D'altra parte, le cache di dati sono spesso più rare su questo tipo di processore. Deriva dal fatto che la posizione temporale dei programmi che utilizzano le tabelle è debole? Inoltre, i registri vettoriali sono spesso più lunghi delle linee della cache. In queste condizioni, passare attraverso una memoria cache intermedia è inutile: tanto vale passare direttamente attraverso i registri vettoriali. Pertanto, i processori vettoriali raramente dispongono di cache.

DOMANDA 23

Quali sono le cause che possono rallentare una pipeline rispetto all'altra?

[Risposta] ci sono delle situazioni, dette hazard, in cui non è possibile iniziare una nuova istruzione ad ogni ciclo di clock. In questi casi, la pipeline si trova in una situazione di stallo, che causa un degrado delle prestazioni.

- Structural hazard → una istruzione deve occupare una risorsa già occupata da un'altra istruzione presente in pipeline. Una possibile soluzione è di progettare opportunamente il set delle istruzioni in modo da evitare o limitare al minimo i casi di conflitto sull'uso delle risorse. Un esempio è che in fase di fetch devo accedere in lettura in memoria per leggere la prossima istruzione da eseguire, mentre in fase di scrittura del risultato devo accedere in scrittura in memoria. In tal caso risolvo il problema utilizzando due memorie, una per le istruzioni e una per i dati.
- Data hazard → un'istruzione ha bisogno di un dato non ancora prodotto da un'istruzione precedente. Come ad esempio:
 - $\text{add } r1, r2, r3 \rightarrow r1 = r2 + r3$
 - $\text{sub } r4, r5, r1 \rightarrow r4 = r5 - r1$

La soluzione può essere usare delle "scorciatoie" a livello hardware, ovvero attraverso dei circuiti di bypass, che riportano l'uscita dello stage di esecuzione all'ingresso del registro che precede lo stage di lettura degli operandi. Un'altra possibile soluzione è l'esecuzione fuori ordine (ovvero non eseguo la sub ma l'istruzione successiva che non ha bisogno del risultato della add). In tal caso devo però stare attenta alle dipendenze tra le istruzioni (renaming dei registri).

- Control hazard → istruzione di salto condizionale di cui non è ancora stata calcolata la condizione. Come ad esempio:
 - Jz r1, avanti → se r1=0, vai avanti
 - Add r2, r3, r4
 - Avanti: sub ..

In tal caso la fase di fetch non sa quale istruzione prelevare, se quella successiva a jz o quella indicata con la label avanti. In tal caso, una possibile soluzione è quella di fare una previsione sul risultato della condizione. La previsione può essere statica, oppure dinamica tramite l'uso dei predittori di salto che memorizzano l'esito dei salti precedentemente incontrati per predire il risultato della condizione. Se si permette l'esecuzione delle istruzioni successive al salto, si parla di esecuzione speculativa. In tal caso deve essere possibile fare il roll back delle istruzioni se la previsione era sbagliata.

DOMANDA 24

Nel caso di numero di thread fisso all'aumentare della dimensione del vettore diventa sempre più conveniente avere più thread, perché?

[Risposta] Quando dobbiamo parallelizzare conviene sempre avere una grande dimensione dei dati. Perché posso sfruttare molti più thread per svolgere le computazioni, con vettori più piccoli che possono essere mantenuti anche in cache potrebbe non essere conveniente avere più thread a causa dell'overhead fisso dato dalla creazione e mantenimento dei thread.

DOMANDA 25

Differenza tra protocollo di coerenza *invalidate* e *update*. In che relazione sono coerenza e consistenza?

I protocolli di coerenza della cache, sia *invalidate-based* che *update-based*, sono utilizzati per garantire che i dati memorizzati nella cache di diversi processori siano coerenti, ovvero che riflettano il valore corrente dei dati nella memoria principale.

Il write update segue la falsa riga dell'approccio write through per l'aggiornamento della memoria principale a seguito di una scrittura. Se un processore esegue una scrittura su un blocco di cache, il blocco modificato viene propagato sul bus in modo che gli altri processori possano aggiornare la loro copia del blocco (se ne hanno una in cache).

Il write invalidate segue la falsa riga dell'approccio write back. Se un processore esegue una scrittura su un blocco di cache, viene inviato un segnale di invalidazione in modo che gli altri processori possano invalidare la loro copia del blocco (se ne hanno una in cache).

La coerenza garantisce che tutti i processori vedano le stesse operazioni nello stesso ordine, mentre la consistenza stabilisce tale ordine. Quindi, in sostanza, la coerenza si occupa di come propagare le scritture, mentre la consistenza si occupa di quando propagarle.

Quale è l'ultimo momento utile per fare l'operazione?

L'ultimo momento utile è prima che i dati vengano letti.

DOMANDA 26

Quale è il vantaggio di non fare subito le operazioni e salvarle in un buffer?

Nella lock e nella unlock, che anticipano una serie di operazioni sui dati condivisi. Salvo in un buffer per non far interrompere il lavoro al processore, quindi li salvo in memoria nell'ultimo momento utile. In questo modo posso sfruttare il fatto che le scritture sono asincrone, quindi posso eseguirle in un momento in cui il processore non è impegnato. Ovviamente questo deve avvenire prima dell'ultimo momento utile, cioè prima che qualcuno tenti di leggere.

DOMANDA 27

Differenza tra throughput, latenza e banda. Cos'è più facile ottenere adesso?

[Risposta] Throughput è la quantità di dati che vengono inviati in un certo intervallo di tempo.

Banda è la quantità massima di dati che possono essere inviati. Latenza è il tempo che trascorre tra l'invio dei dati e l'arrivo. È più facile ottenere la banda, perché non vuol dire ottenere prestazioni migliori, ma solo migliorarle teoricamente.

DOMANDA 28

Out-of-order execution, rinomina dei registri + esempi.

Nell'esecuzione fuori ordine l'esecuzione di ogni istruzione viene fatta iniziale appena possibile, anche se in un ordine diverso da quello stabilito dal programma. Questo può essere fatto se il risultato finale dell'elaborazione non cambia.

In questo modo è possibile eliminare le situazioni di stallo e migliorare le prestazioni.

Per poter avere un'esecuzione fuori ordine dobbiamo però fare attenzione alle dipendenze tra le istruzioni.

Dipendenze sui nomi → che si verificano quando due istruzioni scrivono nello stesso registro, oppure se un'istruzione scrive in un registro letto da un'altra istruzione. Come ad esempio:

Add r1, r2, r3 → $r1 = r2 + r3$

Sub r2, r4, r5 → $r2 = r4 - r5$

Mul r6, r2, r1 → $r6 = r2 * r1$

In tal caso la sub non usa il risultato della add, ma non può essere eseguita prima della add perché modifica il contenuto di r2 che è letto dalla add → dipendenza sui nomi

Le dipendenze sui nomi possono essere eliminate tramite la rinomina dei registri. Quindi, nell'esempio di prima, il risultato della sub può essere scritto in un altro registro non utilizzato (come r7), purché venga modificata anche l'istruzione successiva mul che deve leggere il risultato della sub (non più r2, ma r7)

Add r1, r2, r3 → $r1 = r2 + r3$

Sub r2, r4, r5 → $r7 = r4 - r5$

Mul r6, r2, r1 → $r6 = r7 * r1$

I registri del processore si dividono in architetturali (registri che il compilatore può usare) e fisici (registri della cpu), quindi in fase di esecuzione i registri architetturali devono essere rinominati e mappati nei registri fisici.

Architetture eterogenee e protocolli di coerenza

Le architetture eterogenee sono due:

- SMP (Symmetric multiprocessor) → in tale architettura più processori condividono lo stesso spazio di memoria. Allo scopo di migliorare le prestazioni si introducono delle cache multilivello. Si viene così a creare una gerarchia di memorie, alcune delle quali sono private, altre condivise. I processori accedono alle memorie condivise attraverso un bus condiviso.
- DSM (distributed multiprocessor) → in tali architetture la memoria è suddivisa in banchi, ognuno dei quali è associato ad uno dei processori del sistema. Ogni processore è dotato di uno o più livelli di memoria privata, e a monte viene montata la porzione di memoria condivisa associata a quel processore. Ogni nodo (composto da processore e memoria) è collegato ad una rete di interconnessione che consente la comunicazione con gli altri nodi.

Nelle architetture eterogenee si verifica il problema della cache coherence.

Pertanto, è necessario mettere a disposizione dei processori un meccanismo che consenta di mantenere in ogni istante la correttezza delle informazioni contenute nelle varie cache del sistema, in modo tale che i processori abbiano sempre la stessa visione dello spazio di memoria. Questo viene fatto attraverso i protocolli di coerenza.

Il meccanismo che gestisce la cache coherence deve far sì che ogni lettura di una certa variabile restituisca il valore scritto più di recente per quella variabile.

I protocolli di cache coherence si occupano di mantenere la coerenza dei dati nei blocchi di cache condivisi, cercando di limitare il più possibile la latenza dovuta alla generazione di cache miss dovute alle operazioni di scrittura.

DOMANDA 29

Quali sono i principali problemi che possono nascere, e creare inconsistenza, in una pipeline?

Problema del Write-back.

Prendendo in considerazione la load, si disegni la pipeline e si mostrino i due problemi distinti che si hanno con questa operazione.

1) segnale di write register che deve essere settato ad 1 nel momento in cui si va a fare il write back del register file. Il problema si presenta quando la load arriva allo stage finale dell'esecuzione, in cui non è più presente il segnale di write back → soluzione: prendere il segnale di write back prodotto dalla logica di controllo e propagarlo alla pipeline

2) Per l'indirizzo del registro vale la stessa cosa, cioè deve essere mantenuto

Perché in una memoria cache di dimensioni non elevate è importante la scelta della dimensione del blocco?

In generale, all'aumentare della dimensione del blocco, il miss rate diminuisce.

Nel caso di cache piccole, però, la miss rate aumenta all'aumentare della dimensione del blocco. Questo andamento negativo è dovuto alla località temporale.

La scelta della dimensione del blocco dipende dal principio di località che si vuole sfruttare:

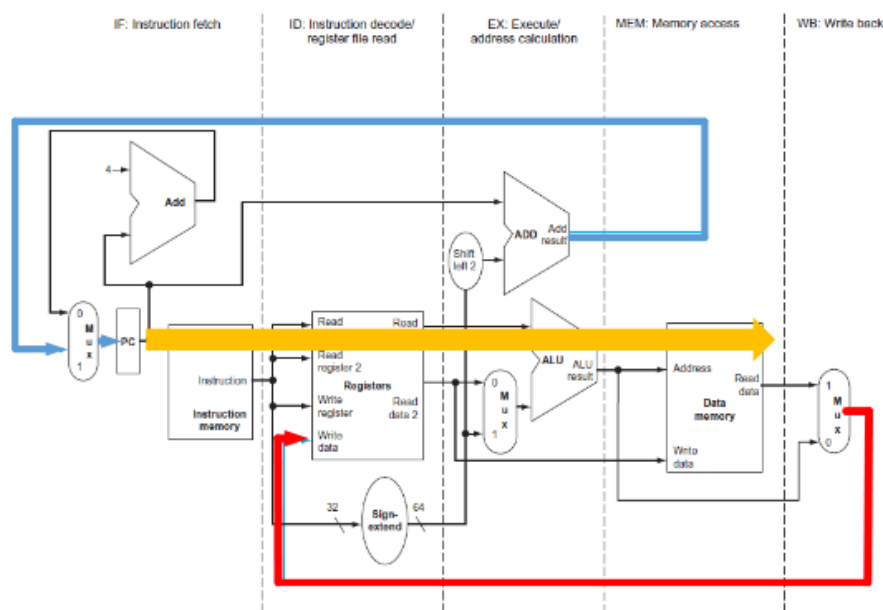
- Per la località spaziale è meglio avere blocchi di grandi dimensioni (infatti, in tal caso con blocchi grandi, vengono caricati in cache più locazioni contigue)

Se il blocco è troppo piccolo non si riesce a sfruttare il principio di località spaziale

- Per la località temporale è meglio avere blocchi di piccole dimensioni (se aumento la dimensione dei blocchi, diminuisce il numero di blocchi che posso memorizzare, ma per sfruttare la località temporale ho bisogno di tanti blocchi in cache)

Se il blocco è troppo grande non si riesce a sfruttare il principio di località temporale

Si disegni un grafico di come va l'andamento delle miss per una cache di 16K in funzione della dimensione di un blocco (vedi disegni domanda 19 o 4)



DOMANDA 30

Vediamo i vari modelli di Multi-threading hardware che ci sono in circolazione.

- Coarse-grained multithreading → ogni volta che un thread causa uno stallo nella pipeline,

faccio un thread switching

- Fine-grained multithreading → i thread si intervallano sempre secondo uno scheduling round-robin. Se ho un thread che non causa stalli, viene comunque eseguito il thread switching, pertanto questo dovrà aspettare il proprio turno per finire la propria esecuzione
- SMT → è un'evoluzione del fine-grained. Ad ogni istante posso eseguire qualsiasi istruzione relativa a qualsiasi thread, in qualsiasi functional unit. In tal caso va mantenuto l'id del thread a cui appartiene l'istruzione in modo che sia possibile committare correttamente i risultati.

Oltre al parallelismo per aumentare le prestazioni è necessario ridurre la comunicazione tra processi.

Queste soluzioni possono avere overhead diverso? C'è overhead?

Sì (overhead dovuto al thread switching), bisogna cambiare il REGISTER FILE, più si eseguono thread in contemporanea meno deve costare il cambio dei registri.

Il register file è una struttura dati all'interno del processore che contiene i registri della CPU, ovvero delle locazioni di memoria molto veloci e di dimensioni ridotte in cui possono essere immagazzinati i dati su cui opera la CPU.

Nel contesto del multithreading, il register file viene utilizzato per implementare il context switch

Cosa sono i "Miss di conflitto"?

Sono CACHE MISS e si hanno quando due blocchi in memoria condividono la stessa posizione in cache.

Come si risolvono?

Ci sono vari modi: (il + efficiente) Usare una cache super-associative, ogni blocco può occupare 4 posizioni. Oppure usare una cache associativa a 4 vie. Oppure aumentare la dimensione della cache aiuta.

Ma migliorare le miss è sempre vantaggioso?

No, dipende dalla complessità che aumenterà il tempo di accesso e la power consumption.

DOMANDA 31 [SEMI-SKIP]

Facciamo riferimento a un multiprocessore, mi deve raccontare perchè ci occupiamo tanto delle write nel multiprocessore

Perchè ogni volta che modifico un valore in un blocco, quando verrà eseguita una miss su questo e verrà richiesto un nuovo blocco non sarà più sufficiente sostituirlo con quello nuovo, ma sarà necessario prima scrivere quello da sostituire in RAM e poi caricare quello nuovo generando molto più overhead rispetto a una normale READ.

Inoltre, le miss dovute alle letture sono mitigate dall'architettura multilivello della cache, mentre le scritture devono comunque essere fatte in memoria prima o poi.

Quindi per migliorare le prestazioni dobbiamo diminuire il numero di scritture.

La presenza delle cache cosa produce come effetti da tener presente nei multiprocessori?

Il problema della cache coherence → è necessario mettere a disposizione dei processori un meccanismo che consenta di mantenere in ogni istante la correttezza delle informazioni contenute nelle varie cache del sistema, in modo tale che i processori abbiano sempre la stessa visione dello spazio di memoria

(?? Le cache miss vengono distribuite in modo non lineare, la cache all'inizio è vuota quindi ci saranno molte miss all'inizio) → non capisco cosa c'entra

Generalmente nei multiprocessori ci sono situazioni in cui alcuni processori diventano lenti, che suggerimento da chi deve progettare il kernel?

Queste situazioni sono quando ad esempio ci sono dei context switch in contemporanea su più processori. In tal caso, si hanno tante miss in cache, e bus e shared memory vengono occupati. Per migliorare le cose bisogna progettare un sistema operativo che abbia uno scheduler che non permetti l'accavallarsi di tali eventi in contemporanea.

Quali sono i principi di base in generale nel costruire acceleratori per sistemi che hanno a che fare con i big data e con sistemi di AI?

Gli ingressi e le uscite sono valori statistici. Quindi ridurre la dimensione dei valori riducendo gli spostamenti in memoria. Non usare un processore general-purpose ma specifici che vanno ad ottimizzare le prestazioni per quei tipi di operazione.

Prendiamo una lente di ingrandimento, abbiamo un processore i7 classico e un acceleratore. Qual è la principale differenza?

Percentuali dell'area di chip utilizzate → le aree di chip utilizzate per l'esecuzione e per l'accesso in memoria sono poche, e il 99% è utilizzato per migliorare le performance

Questo va bene per processori general purpose, ma non per processori special purpose

Domande trovate su cartella anni passati

Cache:

- Why?
- principles of locality
- Cache organization (direct, set, fully associative)
- hit time and miss penalty

- type of misses (3C)
- Replacement policy
- instruction and data cache
- write policy, cache levels
- inclusive and exclusive cache

In an *inclusive* cache model the same data can be present in both the L1 and L2 caches. In an *exclusive* cache, data can be present in only one cache and an address cannot be found in both the L1 and L2 caches at the same time.

- victim cache

GPU:

(Slide di Tecco)

- differences with CPU

La GPU è costituita da più array di SM.

I Streaming Multiprocessors (SM) sono unità di elaborazione presenti nelle schede grafiche (GPU) moderne che consentono l'esecuzione di un grande numero di thread in modo parallelo. Ogni SM è costituito da un insieme di unità di elaborazione, chiamate CUDA Cores, che sono in grado di eseguire le istruzioni in parallelo. Inoltre, ogni SM dispone di un'unità di controllo che coordina l'esecuzione dei thread, gestisce la memoria e l'accesso ai registri, e controlla l'accesso alle risorse condivise.

- heterogeneous architectures

Heterogeneous System Architecture (HSA) is an architecture that combines a Central Processing Unit (CPU) and a Graphics Processing Unit (GPU) to speed up processes.

- memory latency in this architectures

Il problema della latenza nei processori vettoriali si riferisce al fatto che le operazioni su un'ampia gamma di dati richiedono un tempo maggiore per essere eseguite rispetto ad un'operazione su un singolo elemento di dati. Questo perché i processori vettoriali in genere manipolano grandi blocchi di dati in parallelo, ma le istruzioni devono essere eseguite una dopo l'altra in sequenza.

Il problema diventa particolarmente critico quando si lavora con insiemi di dati molto grandi, come quelli utilizzati in applicazioni di calcolo scientifico e di simulazione. In questi casi, il tempo necessario per completare una singola operazione vettoriale può essere di diversi ordini di grandezza più lungo rispetto alla stessa operazione su un singolo elemento.

Per ridurre la latenza nei processori vettoriali, sono state sviluppate diverse tecniche. Una di queste tecniche è l'implementazione di pipeline di esecuzione, che consente di eseguire più istruzioni contemporaneamente in fasi separate. Un'altra tecnica consiste nell'utilizzo di cache di memoria ad alta velocità per ridurre il tempo necessario per accedere ai dati.

Tuttavia, la latenza nei processori vettoriali rimane un problema importante che richiede un'attenzione particolare da parte dei progettisti di hardware e software.

- SIMT

SIMT stands for "Single Instruction Multiple Threads". It is a type of parallel computing architecture in which a single instruction is executed across multiple threads in parallel. This allows for faster processing of data and improved performance in applications that require high levels of parallelism, such as 3D graphics rendering, video processing, and scientific simulations. SIMT is commonly used in graphics processing units (GPUs) and is a key technology in the field of high-performance computing.

- CUDA (Slide di tecco) (thread and memory organization, structure of code)

Multiprocessor:

- Multiprocessor vs Multicomputer,
- Consistency vs Coherence,

- definitions of consistency

Esistono 3 livelli di consistenza:

.Consistenza Processore, in cui le write vengono eseguite all'unlock

.Weak consistency, In cui se processore 1 fa delle write e processore 2 fa altrettanto, al prossimo lock del processore 1 verranno eseguite le write. Ci sarà un breve periodo di inconsistenza.

.Release consistency, in cui si specificano le variabili condivise e si fa lo stesso discorso della weak consistency, ma sulle variabili condivise.

- UMA Architecture,

Uniform memory access (UMA) is a shared memory architecture used in parallel computers. All the processors in the UMA model share the physical memory uniformly. In an UMA architecture, access time to a memory location is independent of which processor makes the request or which memory chip contains the transferred data. Uniform memory access computer architectures are often contrasted with non-uniform memory access (NUMA) architectures. In the NUMA architecture, each processor may use a private cache.

- Cache Coherence with Snooping Cache: write-through, write-back
- (MESI, MOESI, MSI protocols)
- Crossbar interconnection
- omega network
- Cache coherence with directory-based protocol
- Quali sono le possibili miss? Come si possono misurare le percentuali dei vari tipi di miss.
- Differenza tra concorrenza e parallelismo.
- Processori superscalari. Cosa sono, cosa significa esecuzione out of order (slide processor 2 pag 83)
- Dependability e scalability

La dependability si riferisce alla capacità di un sistema di funzionare correttamente, senza interruzioni o guasti, anche in presenza di eventi imprevisti o di malfunzionamenti dei componenti hardware o software. Questo è particolarmente importante per i sistemi critici, come quelli utilizzati nell'aviazione, nella medicina o nelle infrastrutture critiche.

La scalability, invece, si riferisce alla capacità di un sistema di gestire un aumento del carico di lavoro senza perdere prestazioni o efficienza. Ciò significa che il sistema deve essere in grado di adattarsi alle esigenze crescenti dell'utente, senza subire un degrado delle prestazioni o un aumento dei tempi di risposta.

- Che cos'è la scalabilità e che cos'è un collo di bottiglia? Che legame c'è tra i due.
- Quali fattori definiscono una word
- Arm Cortex A8, struttura della memoria (Slide Virtual Memory)
- Livelli di parallelismo, architetturale e applicazione

Il parallelismo architetturale si riferisce alla capacità di elaborazione simultanea di un singolo processore o di più processori. Questo può essere ottenuto attraverso l'uso di più core all'interno di un singolo processore, oppure attraverso l'uso di più processori collegati tra loro, come nel caso di cluster di computer.

Il parallelismo dell'applicazione si riferisce alla capacità di elaborare più operazioni contemporaneamente a livello di singola applicazione. Questo può essere ottenuto attraverso la suddivisione di un'operazione in più attività indipendenti, che possono essere eseguite contemporaneamente su più core o su più processori.

- Come deve struttura il codice il programmatore per sfruttare la parallelizzazione dell'architettura?
- Struttura cache a 2 vie (con disegno). Least Recently Used e Least Frequently Used

Domande Computer Architecture

Raccolta di domande fatte durante le lezioni.

1. [presentazione sui droni] il drone lo costruisco per fare delle elaborazioni (esempio ricostruire la zona con tutte le altezze senza andare a fare misure dirette). Immaginiamo di disporre di un'applicazione in grado di costruire tutto il profilo di un palazzo, ha più senso, dal punto di vista del consumo, utilizzare l'energia per spedire il filmato a terra oppure creare automaticamente le immagini sul drone e inviare le foto, parzialmente elaborate, a terra. (tipo watt per byte o watt per frame)
2. Devo ridurre i conflitti sulla stessa struttura offrendo due copie di tags, una per offrire il tag in caso di miss della cache di livello uno, una per rispondere alle operazioni effettuate sul bus condiviso. Riduco i conflict miss poiché cambiando il tag di un blocco cambio anche il suo indice che poteva fare conflitto in cache.

Se ho due copie, devo aggiornarle quando voglio cambiare queste due copie. Questo è un problema? L'intervallo di tempo in cui devo cambiare i due valori del tag, potrebbe causare un effetto collaterale sulla cache di primo livello o sul bus condiviso.

Il problema è che l'attività tradizionale del primo livello di cache è risolvere le miss del secondo livello e per cercare quale blocco caricare bisogna cercare attraverso il tag.

Quando devo aggiornare i tag devo eseguire un'azione atomica per evitare inconsistenze.

Questa update può essere un problema dal punto di vista delle performance del processore?

Quando devo aggiornare entrambe le copie del tag? In caso di miss nel secondo livello di cache, devo scrivere un nuovo valore nelle copie di tag.

La soluzione è che la condizione per la quale devo aggiornare entrambi i tag è nel caso di miss nella cache di secondo livello, ma quando lo faccio uso il bus condiviso per chiedere una copia del blocco coinvolto nella miss, quindi il bus è occupato da un'operazione richiesta dalla cache di secondo livello, quindi non c'è un drop delle performance.

3. Che side effect potrebbe avere la inclusion property? Nell'ultimo microprocessore intel la inclusion property non è garantita perché può portare a problemi dal punto di vista energetico. Il problema è che se i tre livelli di cache devono avere gli stessi blocchi, la dimensione massima della cache è data dall'ultimo livello di cache, quella più piccola e questo può peggiorare le performance. La soluzione è l'uso di una victim cache che permette di usare una copia locale senza richiederla attraverso il bus condiviso.

La inclusion property è usata per ridurre la complessità del sistema ma questo può portare a problemi di performance.

Come si può risolvere? **Victim cache**, Miss caching places a fully-associative cache between cache and its re-fill path. Misses in the cache that hit in the miss cache have a one cycle penalty, as opposed to a many cycle miss penalty without the miss cache. Victim Caching is an improvement to miss caching that loads the small fully-associative cache with victim of a miss and not the requested cache line.

A victim cache is a hardware cache designed to decrease conflict misses and improve hit latency for direct-mapped caches. It is employed at the refill path of a Level 1 cache, such that any cache-line which gets evicted from the cache is cached in the victim cache. Thus, the victim cache gets populated only when data is thrown out of Level 1 cache. In case of a miss

in Level 1, the missed entry is looked up in the victim cache. If the resulting access is a hit, the contents of the Level 1 cache-line and the matching victim cache line are swapped.

L1

A

VICTIM

C	D (LRU)
---	---------

B

C (LRU)	A
---------	---

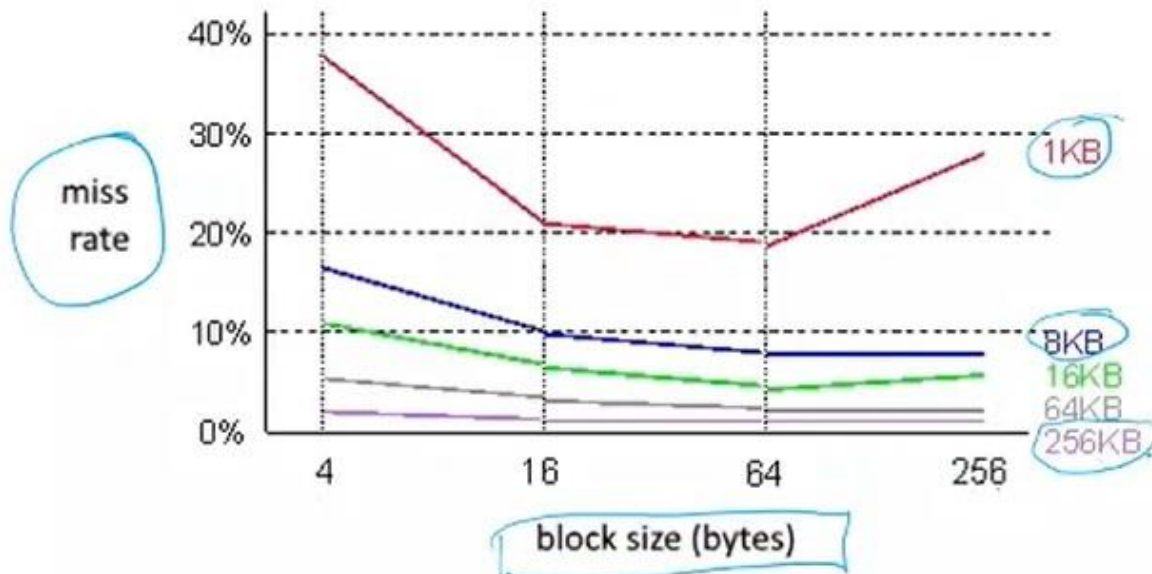
A

C (LRU)	B
---------	---

4. Perché all'interno di un multiprocessore non ho problemi di workload-balancing?

Il workload bilanciato può essere ottenuto automaticamente tramite lo scheduler che offre un task diverso ad ogni processore. L'unico problema è riuscire a fornire un numero di processi pronti tale da fare operare il sistema alla massima utilizzazione e minimizzare la comunicazione tra processi.

5. Andamento miss rate in base alla grandezza del blocco. Vario la dimensione del blocco in bytes (4-16-64-256) e la dimensione della cache in kB (1-8-16-64-256). Perché ha quell'andamento? (**domanda tipica**)



In generale la miss rate va giù quando la dimensione del blocco aumenta. Nel caso della cache da 1 kB invece la miss rate aumenta.

Una soluzione è migliore nel caso cerchiamo *spatial locality*, una nel caso cerchiamo *temporal locality*.

Nel caso di temporal locality voglio avere blocchi piccoli (4 bytes) perché voglio caricarne il più possibile.

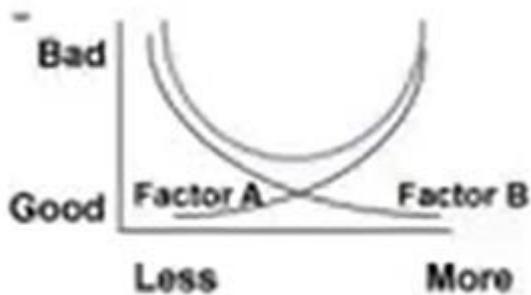
Nel caso di spatial locality vogliamo avere la porzione piú grande di memoria possibile attorno all'indirizzo della miss. (massimizzare la dimensione del blocco)

Tornando al caso della cache da 1 KB, in caso di blocchi da 256 bytes avr  solamente 4 blocchi in cache, che   la migliore situazione per la spatial locality, ma decisamente la soluzione non ottimale per la temporal locality.

Nel caso di blocchi da 4 bytes avr  256 blocchi.

Dobbiamo quindi cercare la dimensione ottimale che solitamente   vicina alla dimensione di una word in cache moltiplicata per quattro.

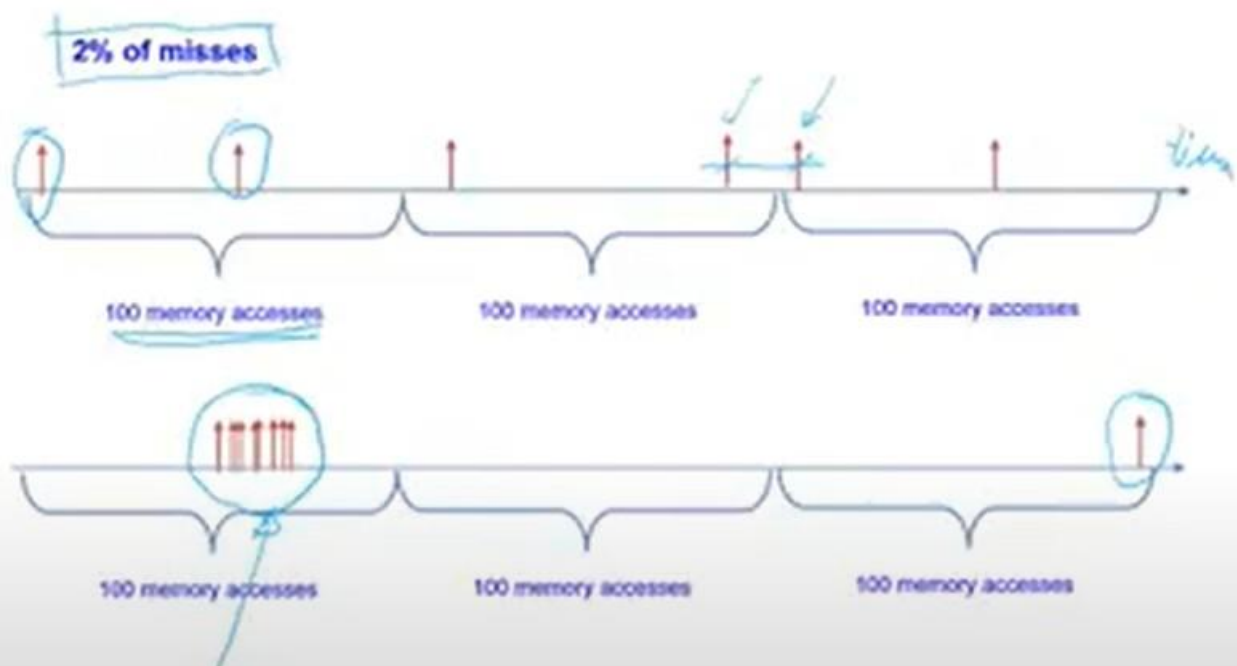
Soluzione = $\text{sizeof}(\text{word}) * 4$.



Il valore ottimo per  potrebbe dipendere da applicazione a applicazione, dato che cambia in base al tipo di accessi che far  l'applicazione in cache.

6. Perch  potrebbero esserci molte miss in un intervallo molto ristretto di tempo e una molto dopo?

unbalanced distribution of misses

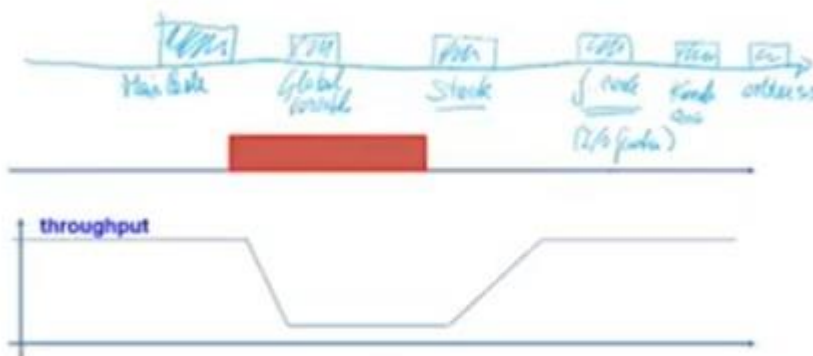


- ♦ chiamate di funzioni (call)
- ♦ cambio di contesto tra processi
- ♦ interrupt (int)

7. Quali sono le possibili aree nelle quali avr  gli accessi? (legata a quella precedente)

- ♦ main code
- ♦ global variables
- ♦ stack
- ♦ codice di funzioni: codice usato per chiamare le librerie, funzioni I/O
- ♦ codice del kernel
- ♦ dati del kernel

unbalanced distribution of misses



20

Conclusioni che possiamo trarre?

Come posso ridurre il numero di miss?

Una buona idea   separare le cache per i dati e le cache per il codice.

2 cache, una per il codice e una per i dati.

Da una parte posso leggere i dati, dall'altra fare il fetch di una nuova istruzione. Posso aumentare il parallelismo tra l'accesso ai dati e l'accesso al codice.

Possiamo organizzare questi due livelli di cache in modi diversi, dato che i meccanismi di localit  spaziale e temporale delle cache dei dati e del codice sono molto diversi.

Direct-mapped cache o set-associative?

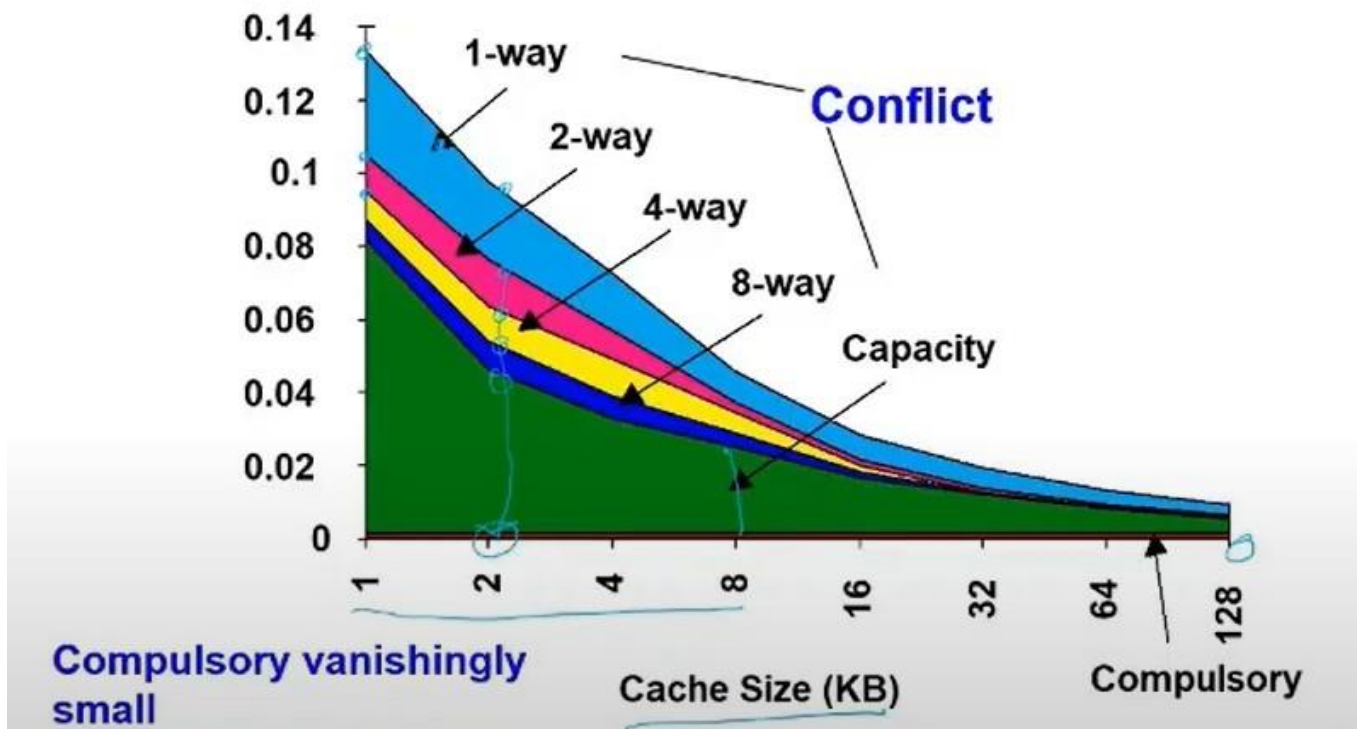
Set-associative, potrebbe esserci dei conflitti dati dal fatto che potrebbero essere eseguite delle istruzioni contenute nello stesso blocco.

Quanti blocchi per set?

4-way associative cache, perch  altrimenti un loop (grande) potrebbe eliminare le copie in cache appartenenti allo stesso loop.

8. Come possiamo valutare la distribuzione delle miss per una cache quando il processore sta eseguendo un particolare programma, riuscendo ad individuare la causa della miss?

3Cs Absolute Miss Rate (SPEC92)



Per la causa *compulsory* bisogna aumentare la dimensione del blocco.

Per *capacity* bisogna aumentare la capacità della cache stessa.

Per le *conflict* si aggiungono n-way alla cache, o utilizzare una fully-associative.

9. Perché i microprocessori attuali usano una cache ad indirizzi fisici al posto di una ad indirizzi virtuali?

Perché due indirizzi virtuali diversi possono mappare lo stesso indirizzo fisico (alias), che invece è univoco.

(detto meglio) Le pagine fisiche possono essere condivise tramite due indirizzi virtuali che puntano allo stesso indirizzo fisico, questo serve per permettere a programmi diversi di condividere porzioni di dati o di codice.

10. Ad un cambio di contesto posso avere molte miss nello stesso momento, come posso evitarlo?

Il problema si verifica quando due o più processori hanno un context switch nello stesso momento. Per risolvere quindi basta non permettere di avere cambi di contesto nello stesso momento.

11. [Domanda presentazione per noi] La sicurezza consuma energia, provate a descrivere la sicurezza a livelli. Si può scegliere il livello di sicurezza che si vuole (l'utente). Vediamo se è ragionevole o no conoscere il livello di sicurezza necessario e vedere il consumo della parte che

potrebbe essere "in piú".

12. Come si può garantire la mutua esclusione in modo atomico? Introdurre una singola istruzione che fa tutto. Se dobbiamo fare la lock e la unlock su un certo valore, senza un'istruzione singola (read, write, load, store), come possiamo fare? Disabilitare le interruzioni. Conoscete un modo per garantire che la test & set sia atomica in un multiprocessore? Il bus é gestito da un controllore (arbiter) che deve selezionare per ogni operazione il processore che può usare il bus. Un modo per risolvere il problema é bloccare il controllore per due operazioni per garantire l'esecuzione atomica e senza interruzioni.

Domande presentazioni

1. [Gruppo 1] Quali sono le differenze tra implementare una smart home solo con la board (Raspberry) o con un server e quindi connessa ad internet. Offline vs Online. In verità la domanda era "é meglio un sistema distribuito o un server centrale?" Possibile prossima domanda: Preferisci la soluzione centralizzata o la soluzione distribuita da un punto di vista del fault-tolerance? Puoi rimarcare i principali difetti di un'architettura distribuita da un punto di vista della progettazione? Si può gestire in single point of failure? Si, tramite repliche.
2. [Gruppo 2] Come determinare la priorità dei dispositivi per un priority arbiter? Potete suggerire un modo per risolvere la *starvation*? La priorità va cambiata ad ogni *miss*.
3. [Gruppo 3] Perché la velocità del clock conta? Perché CoreMark conta? Potete dirmi la situazione del consumo energetico della memoria? Dipende dall'esecuzione del programma o no? E' stabile o aumenta con l'aumentare della frequenza?

La velocità del clock è importante perché rappresenta la frequenza a cui il processore esegue le istruzioni. Più alta è la velocità del clock, più rapidamente il processore può eseguire le istruzioni e completare le operazioni. Tuttavia, aumentare la velocità del clock richiede anche maggiori quantità di energia, il che può comportare un consumo energetico più elevato e un aumento della temperatura del processore.

CoreMark è un benchmark standard che viene utilizzato per misurare le prestazioni dei processori in termini di efficienza, velocità e potenza di elaborazione.

Il consumo energetico della memoria dipende dall'esecuzione del programma e dal tipo di memoria utilizzata. In generale, maggiore è il consumo di energia della memoria, maggiore è il consumo energetico complessivo del sistema. Il consumo energetico della memoria può aumentare con l'aumentare della frequenza, poiché la memoria deve essere in grado di operare a una velocità più elevata per supportare la velocità di clock del processore.

In generale, il consumo energetico del sistema aumenta con l'aumentare della frequenza del clock, poiché maggiori quantità di energia sono necessarie per supportare l'aumento delle prestazioni del processore.

4. [Gruppo 4] Parlare del READ/WRITE protocol della memoria. Ci sono dei problemi con il timing? Potete descrivere la soluzione nella quale il bus é pipelined? Veramente il bus pipelined é organizzato con le fasi sovrapposte. Puoi iniziare una read durante la fase di trasferimento di una write. Un'altra soluzione é lo *splitted bus*.

Il protocollo READ/WRITE della memoria è un protocollo di comunicazione tra un processore e la memoria di sistema. In questo protocollo, il processore può leggere o scrivere dati nella memoria utilizzando un bus di sistema condiviso.

Il protocollo READ/WRITE presenta alcuni problemi di timing, in quanto il processore e la memoria devono lavorare insieme per garantire che i dati siano letti o scritti correttamente e al momento giusto. Ad esempio, se il processore tenta di leggere i dati dalla memoria prima che siano disponibili, o tenta di scrivere i dati nella memoria mentre la memoria sta eseguendo un'altra operazione, possono verificarsi errori di timing che possono causare perdita di dati o comportamenti imprevedibili.

Per superare questi problemi, una soluzione comune è quella di utilizzare un bus pipelined. In un bus pipelined, il bus di sistema è suddiviso in più fasi, e ogni fase è dedicata a un'operazione specifica. Ad esempio, una fase può essere dedicata alla lettura dei dati dalla memoria, mentre un'altra fase può essere dedicata alla scrittura dei dati nella memoria.

In questo modo, le operazioni possono essere svolte in parallelo e senza interferenze, migliorando la velocità di trasmissione dei dati e riducendo il rischio di errori di timing. Tuttavia, l'implementazione di un bus pipelined richiede un'architettura hardware più complessa, che può aumentare i costi e la complessità del sistema.

5. [Gruppo 5] *[presentazione sui droni] il drone lo costruisco per fare delle elaborazioni (esempio ricostruire la zona con tutte le altezze senza andare a fare misure dirette). Immaginiamo di disporre di un'applicazione in grado di costruire tutto il profilo di un palazzo, ha più senso, dal punto di vista del consumo, utilizzare l'energia per spedire il filmato a terra oppure creare automaticamente le immagini sul drone e inviare le foto, parzialmente elaborate, a terra. (tipo watt per byte o watt per frame).
6. [Gruppo 7] Detection e prevenzione degli attacchi.
7. [Gruppo 8] Java processors hardware implementation, energy vs performance
8. [Gruppo 9] Perché l'energia non ha un andamento costante dopo 1.6 GHZ.
9. [Gruppo 10] Come possiamo ottimizzare la power consumption modificando i parametri della cache?
10. [Gruppo 11] Perché l'Intel funziona meglio dell'AMD? Sapete come si ottimizza un'applicazione in base all'architettura?

Domande trovate su cartella anni passati

Cache:

- Why?
- principles of locality
- Cache organization (direct, set, fully associative)
- hit time and miss penalty
- type of misses (3C)
- Replacement policy
- instruction and data cache
- write policy, cache levels
- inclusive and exclusive cache
- victim cache

GPU:

- differences with CPU
- heterogenous architectures,
- memory latency in this architectures
- SIMT
- CUDA (thread and memory organization, structure of code)(slide di tecco)

Multiprocessor:

- Multiprocessor vs Multicomputer,
- Consistency vs Coherence,
- definitions of consistency
- UMA Architecture,
- Cache Coherence with Snooping Cache: write-through, write-back
- (MESI, MOESI, MSI protocols)
- Crossbar interconnection
- omega network
- NUMA architecture
- Cache coherence with directory-based protocol
- Quali sono le possibili miss? Come si possono misurare le percentuali dei vari tipi di miss.

- Differenza tra throughput, latenza e banda. Cos'è più facile ottenere adesso?

[Risposta] Throughput è la quantità di dati che vengono inviati in un certo intervallo di tempo. Banda è la quantità massima di dati che possono essere inviati. Latenza è il tempo che trascorre tra l'invio dei dati e l'arrivo. È più facile ottenere la banda, perché non vuol dire ottenere prestazioni migliori, ma solo migliorarle teoricamente.

- Differenza tra concorrenza e parallelismo.
- Processori superscalari. Cosa sono, cosa significa esecuzione out of order
- Dependability e scalability
- Che cos'è la scalabilità e che cos'è un collo di bottiglia? Che legame c'è tra i due.
- Quali fattori definiscono una word
- Arm Cortex A8, struttura della memoria
- Livelli di parallelismo, architetturale e applicazione
- Come deve struttura il codice il programmatore per sfruttare la parallelizzazione dell'architettura?

- Struttura cache a 2 vie (con disegno). Least Recently Used e Least Frequently Used

Domande esami anni precedenti:

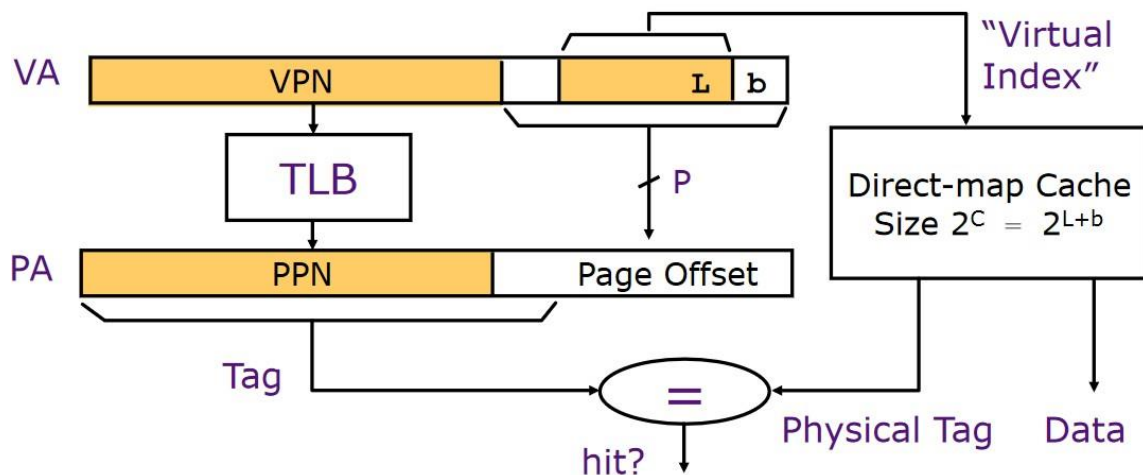
1. Abbiamo visto come organizzare il lavoro in pipeline. Dal punto di vista delle performance quali sono i fattori che possono incidere sul throughput delle ALU?
[Risposta] Stalli causati da attesa di operandi, la soluzione può essere usare delle "scorciatoie" a livello hardware. Possiamo ridefinire la fase di decodifica, oltre alla decodifica ho l'accoppiamento degli operandi di input. Normalmente l'input è o nell'indirizzo o in un registro, adesso possiamo riceverlo anche da una ALU precedente.
Se non usassi questa cosa dovrei inserire uno stallo e aspettare il write-back.
Dopo ha voluto anche sapere il register renaming. Come ci aiuta la cache in questa situazione? Recupero i dati più velocemente. La presenza delle cache introduce vari problemi, ma uno di questi è particolarmente interessante e influenza lo sviluppo del sistema operativo in particolare il multicore, quale? Cache coherence, perché la studiamo? Perché è un bottleneck? Perché è molto più utilizzata e perché il problema fondamentale della cache sono le write. (slide su read e write cache coherence)
2. Perché su un multiprocessore abbiamo introdotto più linee di pipeline? E perché non troviamo processori con 15 pipeline?
[Risposta] Ovviamente aumentiamo linee di pipeline perché aumenta throughput, il problema di averne troppi è che devo accelerare programmi sequenziali per quello che riguardano la CPU. Dovrei aumentare il numero di istruzioni parallele in programmi sequenziali, mentre la maggior parte dei programmi che abbiamo sono sequenziali.
3. Modelli di multithreading per i multiprocessori.
Voleva sapere *coarse-grained* MT, *fine-grained* MT, *SMT*. Soprattutto quali sono i vantaggi dell'avere multithreading.
4. Come si possono ridurre le conflict miss in cache? Poi è passato a chiedere cosa sono le conflict misses.
[Risposta] Aumentare l'associatività.
5. Solitamente abbiamo il primo livello di cache piccolo, i successivi grandi, perché?
[Risposta] Perché vogliamo minimizzare l'access time del primo livello di cache, non ci importa avere un grande numero di hit. Invece i livelli successivi possono avere access time più alti ma devono minimizzare il numero di miss.
6. Come funziona la cache di primo livello in combinazione con la traduzione degli indirizzi? Il primo livello di cache usa indirizzi fisici o virtuali?
[Risposta] Ovviamente uso indirizzi fisici, perché per gli indirizzi virtuali ho un problema. Infatti più di un indirizzo virtuale può riferire lo stesso indirizzo fisico e per questo motivo vado ad usare direttamente i fisici ed evitare di dover aggiungere hardware per fare controlli. Questo porta ad un problema, infatti ora devo prima tradurre l'indirizzo da virtuale a fisico e poi usarlo nella cache. Come possiamo ottimizzare il tempo di accesso?
 - ♦ usare una cache direttamente mappata

- ♦ usare un TLB per la traduzione. (Cosa é un TLB) Piccola cache velocissima (full-associative) che contiene le ultime traduzioni degli indirizzi virtuali.
Come si può ottimizzare questa traduzione? Sono operazioni eseguite in modo sequenziale, come faccio ad eseguirle insieme?
[Risposta] Way-prediction non é giusta come risposta, perché non si usa per diminuire il tempo di accesso ma per diminuire l'energia richiesta.

Risposta giusta.

Virtually Indexed, Physically Tagged Caches

key idea: page offset bits are not translated and thus can be presented to the cache immediately



Index L is available without consulting the TLB

⇒ *cache and TLB accesses can begin simultaneously*

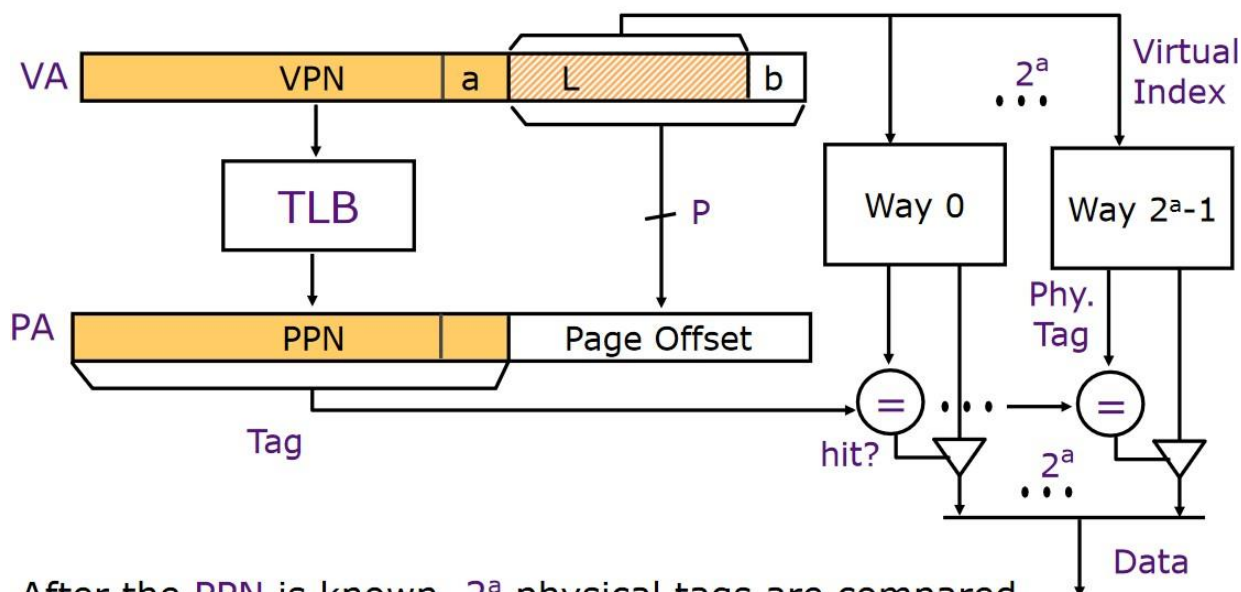
Tag comparison is made after both accesses are completed

Work if Cache Size $\leq$ Page Size ($\rightarrow C (=L+b) \leq P$)

because then all the cache inputs do not need to be translated

Adapted from Arvind and Krste's MIT Course 6.823 Fall 05

Virtually-Indexed Physically-Tagged Caches: Using Associativity for Fun and Profit



After the PPN is known, 2^a physical tags are compared

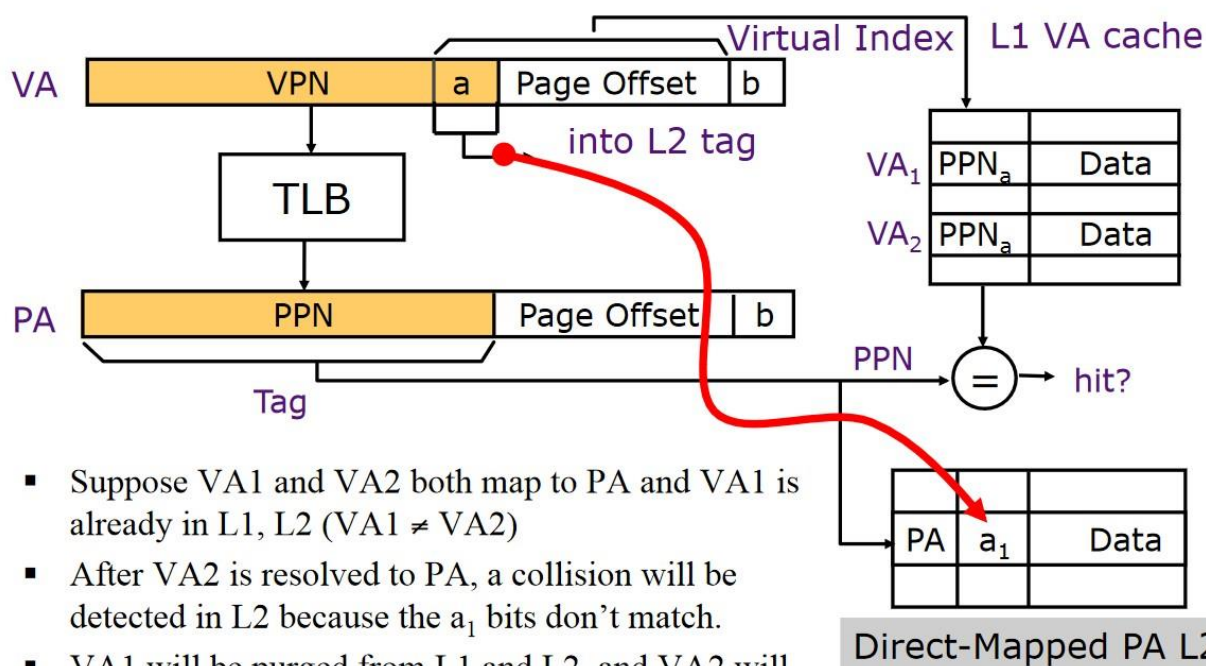
Increasing the associativity of the cache reduces the number of address bits needed to index into the cache -

Work if: $\text{Cache Size} / 2^a \leq \text{Page Size}$

$$(\rightarrow C \leq P + A)$$

Anti-Aliasing Using Inclusive Direct Mapped L2: MIPS R10000-style

Once again, ensure the invariant that only one copy of physical address is in virtually-addressed L1 cache at any one time. The physically-addressed L2, which includes contents of L1, contains the missing virtual address bits that identify the location of the item in the L1.



- Suppose VA1 and VA2 both map to PA and VA1 is already in L1, L2 ($\text{VA1} \neq \text{VA2}$)
- After VA2 is resolved to PA, a collision will be detected in L2 because the a_1 bits don't match.
- VA1 will be purged from L1 and L2, and VA2 will be loaded $\Rightarrow$ no aliasing !

Direct-Mapped PA L2

(could be associative too, just need to check more entries)

Adapted from Arvind and Krste's MIT Course 6.823 Fall 05

7. Perché si usano due cache di livello 1, una per le istruzioni e una per i dati?
[Risposta] Perché nella pipeline in questo modo posso fare il fetch di un'istruzione in contemporanea alla lettura di un dato.
8. Dal punto di vista del consumo energetico è più importante l'energia o la potenza dissipata? Che importanza ha controllare il picco massimo di potenza di un processore?
L'energia dissipata è quella che riscalda il nostro sistema. Il picco di potenza da fastidio perché $P=VI$, se accendo contemporaneamente 4 vie devo fornire corrente e in questo momento ho un picco di potenza dato dalla corrente di 4 vie accese. Si crea un *hotspot*, picco di potenza localizzato e breve che non si può controllare con la ventola o un altro di sistema di raffreddamento. I sistemi embedded quindi devono mantenere bassa sia la potenza che l'energia.
9. **[SKIPPATA]** Quali sono le regole più importanti per individuare il parallelismo all'interno di CUDA. [Risposta] Parallelismo più importante che ho con CUDA è il *data-level* ma anche *multithread*.
regola 1: *overhead della gestione del parallelismo (distribuzione del carico sui vari thread) è marginale rispetto alla elaborazione del dato* [cosa è l'overhead nel caso della cpu? cambi di contesto, comunicazione tra thread, scrittura]
regola 2: *massimizzare l'uso dei processori*
10. Andamento del tempo di esecuzione totale al variare della dimensione del blocco con capacità della cache.
[Risposta] Voleva sapere località temporale e spaziale.
11. Quale è il metodo più veloce che possiamo avere per capire le prestazioni di un sistema?
[Risposta] Tool di debug per trovare il bottleneck, trovo la componente più utilizzata, faccio un benchmark su queste componenti.
12. A partire da una organizzazione funzionale di una CPU siamo arrivati ad una organizzazione a pipeline, perché la pipeline mi permette di aumentare le prestazioni e come faccio a vedere che effettivamente aumentano?
[Risposta] Aumento utilizzazione dividendo le fasi di utilizzo della CPU e potendo iniziare ad eseguire un'istruzione prima che l'ultima sia completamente finita. Il parametro che posso guardare è il numero di operazioni committate al secondo (per esempio). Devo mantenere cariche quante più ALU occupate possibile.
13. Quali sono le soluzioni che si possono utilizzare per un processore vettoriale e quali sono i punti di sovrapposizione tra un processore vettoriale e una GPU? Uso della cache in questo tipo di architettura?

Un processore vettoriale è un tipo di processore specializzato nella manipolazione di grandi quantità di dati in forma vettoriale, ad esempio nell'esecuzione di operazioni matematiche su matrici o nella grafica 3D. Esistono diverse soluzioni hardware che possono essere utilizzate per implementare un processore vettoriale, tra cui:

- Processori vettoriali dedicati: sono chip specializzati progettati specificamente per eseguire operazioni vettoriali. Questi processori sono utilizzati in ambiti come la grafica 3D, la simulazione fisica, la modellizzazione matematica e altri tipi di elaborazione scientifica.
- Processori multi-core: i processori moderni utilizzano l'architettura multi-core per aumentare le prestazioni. I core possono essere utilizzati per eseguire

operazioni vettoriali in parallelo, migliorando le prestazioni complessive del sistema.

- Acceleratori hardware: molte schede madri per PC e server supportano acceleratori hardware come le GPU, che possono essere utilizzate per eseguire operazioni vettoriali.

Esistono alcuni punti di sovrapposizione tra un processore vettoriale e una GPU, poiché entrambi sono progettati per eseguire operazioni matematiche in parallelo. Tuttavia, le GPU sono progettate principalmente per l'elaborazione grafica, mentre i processori vettoriali sono progettati per l'elaborazione matematica generale. Inoltre, le GPU sono progettate per eseguire operazioni in parallelo su grandi set di dati, mentre i processori vettoriali sono progettati per eseguire operazioni vettoriali su set di dati più piccoli.

La cache è un componente critico nell'architettura di un processore vettoriale, poiché può consentire di ridurre i tempi di accesso alla memoria principale e migliorare le prestazioni. I processori vettoriali utilizzano spesso una cache a due livelli, con una cache L1 vicina ai core e una cache L2 condivisa tra i core. La cache L1 contiene dati utilizzati frequentemente, mentre la cache L2 contiene dati meno frequenti ma ancora importanti. L'uso della cache dipende dal tipo di operazioni eseguite dal processore vettoriale e dalla dimensione dei dati elaborati.

14. Quali sono le cause che possono rallentare una pipeline rispetto all'altra?

[Risposta] Voleva sapere gli hazard.

15. Nel caso di numero di thread fisso all'aumentare della dimensione del vettore diventa sempre più conveniente avere più thread, perché?

[Risposta] Quando dobbiamo parallelizzare conviene sempre avere una grande dimensione dei dati. Perché posso sfruttare molti più thread per svolgere le computazioni, con vettori più piccoli che possono essere mantenuti anche in cache potrebbe non essere conveniente avere più thread a causa dell'overhead fisso dato dalla creazione e mantenimento dei thread.

16. Differenza tra protocollo di coerenza *invalidate* e *update*. In che relazione sono coerenza e consistenza?

[Risposta] La prima è la differenza tra write-back e write-invalidate. La seconda: *coherence* quando (temporalmente) applicare tecniche di consistenza, consistenza come salvare dati in modo consistente. Quale è l'ultimo momento utile per fare l'operazione? Quale è il vantaggio di non fare subito le operazioni e salvarle in un buffer? Nella lock e nella unlock, che anticipano una serie di operazioni sui dati condivisi. Salvo in un buffer per non far interrompere il lavoro al processore.

17. Come possiamo implementare a livello hardware la mutua esclusione?

[Risposta] La lock ha un parametro, la variabile che vado a gestire in mutua esclusione e che impedisce agli altri di entrare in quella sezione critica. Come faccio ad impedire agli altri di entrare ma far entrare a me che posso? Prima leggo lo stato (lettura) della variabile, vedo lo stato, setto occupato (scrittura) e entro nella sezione critica. Ma un altro processore potrebbe infilarsi tra la lettura e la scrittura, quindi ho bisogno di un meccanismo più a basso livello.

Questo meccanismo è la TEST AND SET o disabilito interruzioni.

- **Differenza tra True sharing e false sharing:**

Il true sharing si verifica quando due o più thread accedono contemporaneamente alla stessa posizione di memoria nella cache, causando un conflitto di accesso. Questo problema si verifica quando i thread condividono effettivamente lo stesso dato, e quindi la cache deve sincronizzare gli accessi per garantire la corretta coerenza dei dati. Il true sharing può causare una significativa riduzione delle prestazioni del sistema a causa della maggiore latenza dovuta alla sincronizzazione della cache.

Il false sharing, d'altra parte, si verifica quando due o più thread accedono a posizioni di memoria diverse nella stessa linea di cache. Anche se i dati sono diversi, la cache deve sincronizzare gli accessi a causa della condivisione della stessa linea di cache. Questo problema può causare un'eccessiva invalidazione della cache e una maggiore latenza a causa della sincronizzazione della cache, anche se i dati in questione sono in realtà indipendenti.

